

Documentation Complète — Projet Ecommerce

Page 1/16 · Vue d'ensemble et objectifs

Cette documentation décrit en profondeur l'application **Projet_ecommerce**, sa construction technique, ses modules métier, et son fonctionnement côté **client** et **administrateur**. Le document est produit à partir de l'analyse directe du code source frontend + backend.

Frontend React + TypeScript + Vite	Backend Node.js + Express + Zod	Base de données MySQL / MariaDB	Paiement Airtel, Maisha Pay, WhatsApp, Livraison
--	--	--	--

But fonctionnel

- Permettre la découverte de produits, la constitution d'un panier, la création de commandes et leur suivi.
- Offrir un back-office administrateur pour gérer catalogue, catégories, commandes, transactions, utilisateurs, ventes et indicateurs.
- Intégrer des canaux de paiement hybrides adaptés au contexte local (mobile money et assistance WhatsApp).

Périmètre couvert

- Architecture technique, routage, sécurité (JWT + rôles), API REST, modèle de données.
- Parcours client et admin détaillés, logique de stock, notifications, facturation PDF, chatbot.
- Déploiement Railway, variables d'environnement, exploitation et recommandations production.

Page 2/16 · Architecture globale

L'application suit une architecture **SPA + API** avec séparation claire entre interface utilisateur et logique métier serveur.

Couche présentation (Client Web)

- **React Router** Navigation par routes protégées.
- **AuthContext** Stockage token + utilisateur.
- **Lib API** Appels HTTP avec timeout, headers et gestion d'erreurs.
- **UI** Header/Footer/Chatbot + pages métier.

Couche métier (API Express)

- Validation des entrées avec Zod.
- Contrôle d'accès par middleware JWT et rôles.
- Transactions SQL pour commandes/paiements.
- Génération de factures et rapports PDF avec Puppeteer.

Flux principal

Navigateur → /api/* (fetch) → Express → MySQL pool → réponse JSON/HTML/PDF

Points de fiabilité

- Pool DB avec retries sur erreurs transitoires.
- Health endpoints /api/health et /api/health/db.
- Support Railway URL + variables individuelles.
- Fallback SPA pour servir le build frontend.

Page 3/16 · Frontend: fondations techniques

Boot applicatif

- **main.tsx** instancie React, BrowserRouter et AuthProvider.
- **App.tsx** définit toutes les routes et protections d'accès.
- **Layout.tsx** encapsule Header, Footer, Outlet, Chatbot.

Gestion de session

- Token JWT stocké dans `localStorage(auth_token)`.
- Utilisateur courant dans `localStorage(auth_user)`.
- AuthContext expose `login/register/logout/refreshMe`.

Routage sécurisé

Guard	Règle	Exemples
RequireAuth	Utilisateur connecté	commandes, notifications
RequireAdmin	Rôle admin	dashboard, catégories, ventes, utilisateurs
RequireClient	Rôle client	panier, adresses, paiement

Client HTTP

- Base URL dynamique selon `VITE_API_URL` et mode production.
- Timeout configurable, abort controller, messages d'erreurs contextualisés.
- Helpers standards: GET, POST, PUT, DELETE.

Page 4/16 · Backend: structure et middlewares

Composants backend

- **server.js** lance l'écoute HTTP.
- **index.js** contient routes, services et middleware.
- **auth.js** signature/validation JWT et contrôle de rôle.
- **db.js** stratégie de connexion DB et résilience.

Sécurité et contrôle d'accès

- JWT obligatoire pour les routes sensibles.
- Roles stricts: `client` / `admin`.
- Protection contre actions interdites (ex: suppression de son propre compte admin).

Middlewares et robustesse

- Parser JSON avec gestion explicite des JSON invalides.
- CORS multi-origines paramétrable via variable d'environnement.
- Logger API pour traçabilité des requêtes.
- SMTP vérifié au démarrage avec retry.

Services internes

- Envoi facture (email + trace notification WhatsApp).
- Services paiement (Maisha Pay / Airtel Money).
- Génération PDF facture et rapport commandes.

Page 5/16 · Modèle de données (MySQL)

Le socle SQL couvre le cycle complet e-commerce: catalogue, panier, commande, paiement, ventes, CRM et analytics.

Domaine	Tables principales	Rôle métier
Identité	utilisateurs, adresses	Profils, rôles, coordonnées, livraison/facturation
Catalogue	produits, categories, bannieres, historique_prix	Offre marchande, merchandising, audit des prix
Achat	paniers, panier_articles, commandes, commande_details	Panier et conversion en commande
Paiement	transactions_paiement	Traçage des transactions externes
Exploitation	notifications, ventes, visites, avis	Suivi, alertes, BI, qualité perçue

Principes relationnels

- Une commande possède plusieurs lignes de détail.
- Une vente est corrélée à une commande validée.
- Les changements de prix sont historisés.
- Les notifications matérialisent les événements opérationnels.

Intégrité métier

- Contrôles de stock avant création de commande.
- Restauration stock sur annulation de livraison.
- Idempotence partielle sur création vente/transaction.

Page 6/16 · Authentification, autorisation et sécurité

Inscription / connexion

- Inscription: validation email + mot de passe, hash bcrypt côté API.
- Connexion: vérification identifiants puis émission token JWT 7 jours.
- Route /api/me: récupération profil courant.

Rôles et séparation des responsabilités

Client

- Consulte produits/catégories/avis.
- Gère panier local et adresses.
- Passe commande et initie paiement.

Admin

- Gère catalogue et catégories.
- Valide paiements/livraisons.
- Pilote ventes, utilisateurs, transactions et KPI.

Mesures de sécurité

- Endpoints critiques protégés par double middleware auth + rôle.
- Validation stricte des payloads par Zod.
- Normalisation des URL d'images pour réduire les entrées invalides.
- Aucun stockage de PIN Airtel durable côté front.

Page 7/16 · Parcours Client détaillé

1) Découverte et recherche produit

- Filtres: catégorie, recherche textuelle, intervalle de prix, tri.
- Affichage stock faible, images normalisées + fallback.

2) Panier

- Stockage local `cart_v1` (localStorage).
- Actions: ajouter, modifier quantité, retirer, vider panier.
- Calcul sous-total/total côté client.

3) Checkout et paiement

- Modes: Maisha Pay, Airtel, WhatsApp, livraison.
- Création commande + détails via API unique d'initiation.
- Si Airtel: phase éventuelle de validation PIN.
- Si WhatsApp: redirection vers conversation préremplie.

4) Post-achat

- Consultation commandes et détails article par article.
- Accès facture HTML/PDF et partage.
- Ajout d'avis et gestion d'adresses.

Page 8/16 · Parcours Administrateur détaillé

Dashboard stratégique

- Ventes 60 jours et ventes globales.
- Clients connectés selon fenêtre d'activité.
- Distribution produits par catégorie.
- Histogramme visites par heure (24h).

Pilotage opérationnel

- Validation paiement commande (passage à payé).
- Marquage livraison livrée/annulée avec effets de stock.
- Consultation des transactions financières.
- Gestion des rôles utilisateurs et suppression contrôlée.

Production documentaire

- Rapport PDF global des commandes.
- Factures PDF individuelles avec QR code.

Back-office catalogue

- CRUD produits et catégories.
- Bannières promotionnelles.
- Historique des changements de prix.

Page 9/16 · Catalogue, stock et contenus

Produits

- Champs métier: nom, prix, stock, marque, catégorie, image, description.
- Création: détection de doublon logique et consolidation du stock.
- Mise à jour: insertion dans historique_prix si changement tarifaire.
- Suppression: endpoint admin dédié.

Catégories

- Gestion admin complète (ajout, modification, suppression).
- Statut actif exploitable pour filtrer la publication.

Bannières

- Création admin avec image validée et lien optionnel.
- Activation/désactivation via attribut booléen.

Gestion des images

- Formats acceptés: URL HTTP(S), chemin /media, data:image base64.
- Normalisation front + back pour cohérence.
- Fallback visuel pour éviter les blocs cassés.

Page 10/16 · Paiements et orchestration transactionnelle

Pipeline de paiement unifié

La route POST /api/paiements/initier centralise la création de commande et la logique de branchement selon la méthode choisie.

Méthode	Comportement	Sortie
Maisha Pay	Appel API externe + enregistrement transaction en attente	paymentUrl
Airtel	Push USSD simulé/initié + suivi PIN	pending_pin
WhatsApp	Commande créée + URL wa.me préremplie	whatsappUrl
Livraison	Commande en attente de validation manuelle	success

Garanties métier

- Transaction SQL lors de création commande/détails.
- Contrôle stock article par article avant validation.
- Nettoyage panier DB utilisateur après transformation en commande.
- Création vente uniquement quand paiement validé.

Webhook Maisha Pay

- Mise à jour statut commande.
- Mise à jour transaction_paiement.
- Déclenchement facture et notification.

Page 11/16 · Commandes, livraison, factures et ventes

Cycle de vie commande

- **en_attente** → **paye** (validation paiement).
- **traitement** → **expedie** → **livre** (logistique).
- **annule** possible avec restauration de stock.

Règles critiques

- Passage à payé déclenche création vente (idempotente).
- Livré peut archiver/supprimer la commande active selon logique implémentée.
- Échec paiement et annulation créent des notifications opérationnelles.

Facturation

- Facture HTML consultable par référence commande.
- Export PDF via Puppeteer (A4, impression propre).
- QR code intégré pointant vers URL facture en ligne.

Rapport global PDF

- Accessible admin, synthétise total commandes, CA, livraisons.
- Liste détaillée des commandes pour audit.

Page 12/16 · Notifications, chatbot et communication

Notifications

- Événements alimentés côté backend (paiement, panier, livraison, erreurs, WhatsApp).
- Affichage frontend avec marquage lu/non-lu.
- Badge dynamique dans l'entête.

Chatbot

- Interface flottante persistante sur toutes les pages via Layout.
- Backend analyse message et répond selon règles métier:
- Recherche produits, listing catégories, suivi commande par référence CMD-XXX.
- Réponses institutionnelles et fallback intelligent.

Canaux externes

- WhatsApp support dans Header/Footer/Panier/Paiement.
- Email SMTP pour envoi des factures.
- Partage de facture via lien instantané.

Observabilité utilisateur

- Traçage des visites (IP + route) consultable côté admin.
- Historique prix et avis pour pilotage qualité.

Page 13/16 · Qualité logicielle et résilience

Validation et robustesse

- Schémas Zod pour entrées API sensibles.
- Gestion d'erreurs explicites sur toutes les routes majeures.
- Retry SQL sur erreurs transitoires (ETIMEDOUT, ECONNRESET...).

Résilience réseau

- Configuration DNS améliorée pour éviter les timeouts Node/Windows.
- SMTP vérifié et reconnecté avec stratégie de tentatives.
- Timeout frontend configurable pour fetch API.

Consistance des médias

- Normalisation image centralisée front/back.
- Support local upload converti pour portabilité en ligne.
- Fallback visuel universel.

Limites actuelles identifiées

- Panier majoritairement localStorage côté front (complété par API panier côté backend).
- Certaines routes notifications (unread-count / mark-as-read) dépendent d'implémentation complète backend.
- Multiplicité des modes paiement nécessite supervision opérationnelle stricte.

Page 14/16 · Déploiement Railway (production)

Pipeline

- Build: `npm run railway:build`
- Start: `npm run start (node backend/server.js)`
- Healthcheck: `/api/health`

Variables recommandées

Variable	Exemple	Usage
JWT_SECRET	long secret	Signature JWT
MYSQL_URL	<code>\${{MySQL.MYSQL_URL}}</code>	Connexion DB prioritaire
CORS_ORIGIN	<code>https://ton-domaine</code>	Politique CORS
DB_SSL	<code>true/false</code>	TLS DB selon provider
NODE_ENV	<code>production</code>	Mode runtime

Correctif production appliqué

- Détection runtime Railway et gestion correcte des hôtes `railway.internal`.
- Blocage du fallback localhost en production cloud.
- Message d'erreur explicite si host DB local détecté sur Railway.

Checklist go-live

- Tester `/api/health/db` après déploiement.
- Vérifier CORS et URL frontend.
- Valider paiement sandbox avant mode réel.

Endpoints principaux

Domaine	Routes clés
Health	/api/health, /api/health/db, /api/stats
Auth	/api/auth/register, /api/auth/login, /api/me
Catalogue	/api/categories, /api/produits, /api/bannieres, /api/historique-prix
Achat	/api/panier, /api/commandes, /api/commandes/:id/details
Paie me nt	/api/paiements/initier, /api/paiements/valider-pin, /api/paiements/maishapay/callback
Admin	/api/admin/dashboard, /api/admin/transactions, /api/admin/ventes, /api/admin/utilisateurs
Service	/api/chatbot, /api/factures/:reference, /api/admin/rapport-commandes/pdf

Convention de sécurité API

- Public: consultation générale (produits, categories, avis, bannieres, health).
- Client: panier, adresses, paiements, commandes personnelles.
- Admin: gouvernance complète (stocks, utilisateurs, statuts, reporting).

Codes de statut usuels

- 200/201: succès opérationnel.
- 400: erreur de validation ou payload invalide.
- 401/403: authentification ou droits insuffisants.
- 404: ressource absente.
- 500: erreur interne / intégration externe indisponible.

Page 16/16 · Exploitation, tests et feuille de route

Procédures d'exploitation

- Démarrage local: frontend + API via scripts npm.
- Surveillance: health endpoints, logs paiement, logs notifications.
- Backup DB recommandé avant opérations massives admin.

Tests recommandés

- Tests fonctionnels bout-en-bout: inscription → panier → paiement → facture.
- Tests de rôle: blocages admin/client selon route.
- Tests de non-régression sur stock et transitions commande.
- Tests d'intégration Maisha/Airtel en sandbox.

Améliorations prioritaires

- Unifier pleinement panier front et panier SQL backend.
- Compléter endpoints notifications pour unread-count/mark-as-read.
- Introduire pagination serveur sur gros volumes (produits, ventes, commandes).
- Ajouter journal d'audit admin (qui a changé quoi et quand).

Conclusion

L'application est déjà structurée pour un usage réel avec une base technique solide. Les modules client et admin sont clairement séparés, les flux critiques sont transactionnels, et les canaux de paiement/communication sont adaptés à un contexte e-commerce moderne.